NAME : DARSSHINI BABU RAJU
ROLL NO : 2025115144

1)Simulate BFS and DFS traverse all for the given graph.

```
#include <stdio.h>

int graph[20][20], visited[20], n;

void dfs(int v) {
    visited[v] = 1;
    printf("%d ", v);

    for (int i = 0; i < n; i++)
        if (graph[v][i] && !visited[i])
            dfs(i);
}

void bfs(int start) {
    int q[100], front = 0, rear = 0;

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    visited[start] = 1;
    q[rear++] = start;

    while (front < rear) {
        int v = q[front++];
        printf("%d ", v);

        for (int i = 0; i < n; i++) {
            if (graph[v][i] && !visited[i]) {
                visited[i] = 1;
                q[rear++] = i;
            }
        }
    }
```

```
}

int main() {
    int e, u, v, start;

    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &e);

    printf("Enter edges (u v):\n");
    for (int i = 0; i < e; i++) {
        scanf("%d%d", &u, &v);
        graph[u][v] = graph[v][u] = 1;
    }

    printf("Enter starting vertex: ");
    scanf("%d", &start);

    printf("BFS: ");
    bfs(start);

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    printf("\nDFS: ");
    dfs(start);

    return 0;
}
```

OUTPUT :

```
Enter vertices and edges: 6 7
Enter edges (u v):
0 1
0 2
1 3
1 4
2 4
3 5
4 5
Enter starting vertex: 0
```

BFS: 0 1 2 3 4 5
DFS: 0 1 3 5 4 2

2) Find out minimum spanning tree for the given graph using prims and Kruskals algorithm.

```
#include <stdio.h>
#define INF 99999

int main() {
    int n, m, g[20][20], used[20] = {0};
    int parent[20], key[20];
    int eu[100], ev[100], ew[100], temp;
    int p[20], cost = 0, count = 0;

    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &m);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            g[i][j] = INF;

    printf("Enter edges (u v weight):\n");
    for (int i = 0; i < m; i++) {
        scanf("%d%d%d", &eu[i], &ev[i], &ew[i]);
        g[eu[i]][ev[i]] = g[ev[i]][eu[i]] = ew[i];
    }

    for (int i = 0; i < n; i++)
        p[i] = i;

    for (int i = 0; i < m - 1; i++)
        for (int j = 0; j < m - i - 1; j++)
            if (ew[j] > ew[j + 1]) {
                temp = ew[j]; ew[j] = ew[j + 1]; ew[j + 1] = temp;
                temp = eu[j]; eu[j] = eu[j + 1]; eu[j + 1] = temp;
                temp = ev[j]; ev[j] = ev[j + 1]; ev[j + 1] = temp;
            }
```

```
printf("\nKruskal MST:\n");
cost = 0;
for (int i = 0; i < m && count < n - 1; i++) {
    int a = eu[i], b = ev[i];
    while (p[a] != a) a = p[a];
    while (p[b] != b) b = p[b];

    if (a != b) {
        printf("%d - %d : %d\n", eu[i], ev[i], ew[i]);
        p[a] = b;
        cost += ew[i];
        count++;
    }
}
printf("Cost = %d\n", cost);

for (int i = 0; i < n; i++) {
    key[i] = INF;
    parent[i] = -1;
    used[i] = 0;
}

key[0] = 0;
cost = 0;

printf("\nPrim MST:\n");
for (int k = 0; k < n; k++) {
    int u = -1;
    for (int i = 0; i < n; i++)
        if (!used[i] &&
            (u == -1 || key[i] < key[u]))
            u = i;

    if (u == -1 || key[u] == INF) {
        printf("Graph is disconnected\n");
        return 0;
    }

    used[u] = 1;
```

```
        if (parent[u] != -1) {
            printf("%d - %d : %d\n",
                   parent[u], u, key[u]);
            cost += key[u];
        }

        for (int v = 0; v < n; v++)
            if (!used[v] && g[u][v] < key[v]) {
                key[v] = g[u][v];
                parent[v] = u;
            }
    }

    printf("Cost = %d\n", cost);
    return 0;
}
```

OUTPUT :

```
0 1 1
0 2 7
1 2 5
1 3 4
1 4 3
2 4 6
3 4 2

Kruskal MST:
0 - 1 : 1
3 - 4 : 2
1 - 4 : 3
1 - 2 : 5
Cost = 11

Prim MST:
0 - 1 : 1
1 - 4 : 3
4 - 3 : 2
1 - 2 : 5
Cost = 11
```

3) Find minimum spanning tree.

```
#include <stdio.h>

struct Edge {
    int u, v, w;
};

struct Edge e[100], t;
int parent[100];

int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]);
    return parent[x];
}

int main() {
    int n, m, cost = 0, count = 0;

    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &m);

    printf("Enter u v weight for each edge:\n");
    for (int i = 0; i < m; i++)
        scanf("%d%d%d", &e[i].u, &e[i].v, &e[i].w);

    for (int i = 0; i < n; i++)
        parent[i] = i;

    for (int i = 0; i < m - 1; i++)
        for (int j = 0; j < m - i - 1; j++)
            if (e[j].w > e[j + 1].w) {
                t = e[j];
                e[j] = e[j + 1];
                e[j + 1] = t;
            }
```

```
    printf("Minimum Spanning Tree:\n");

    for (int i = 0; i < m && count < n - 1; i++) {
        int a = find(e[i].u);
        int b = find(e[i].v);

        if (a != b) {
            printf("%d - %d : %d\n",
                   e[i].u, e[i].v, e[i].w);
            parent[a] = b;
            cost += e[i].w;
            count++;
        }
    }

    if (count == n - 1)
        printf("Minimum cost = %d\n", cost);
    else
        printf("Graph is disconnected\n");

    return 0;
}
```

OUTPUT :

```
Enter vertices and edges: 4 5
Enter u v weight for each edge:
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4
Minimum Spanning Tree:
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
Minimum cost = 19
```

4) Dijkstra's algorithm.

```c
#include <stdio.h>
#define INF 99999

int main() {
    int n, m, g[20][20], d[20], used[20];
    int u, v, w, source;

    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &m);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            g[i][j] = (i == j) ? 0 : INF;

    printf("Enter edges (u v weight):\n");
    for (int i = 0; i < m; i++) {
        scanf("%d%d%d", &u, &v, &w);
        g[u][v] = w;
        g[v][u] = w;
    }

    printf("Enter source vertex: ");
    scanf("%d", &source);

    for (int i = 0; i < n; i++) {
        d[i] = INF;
        used[i] = 0;
    }
    d[source] = 0;

    for (int k = 0; k < n; k++) {
        int x = -1;
        for (int i = 0; i < n; i++)
            if (!used[i] &&
                (x == -1 || d[i] < d[x]))
                x = i;

        if (x == -1 || d[x] == INF)
            break;
```

```
        used[x] = 1;

        for (int i = 0; i < n; i++)
            if (g[x][i] != INF && !used[i] &&
                d[x] + g[x][i] < d[i])
                d[i] = d[x] + g[x][i];
    }

    printf("Shortest distances:\n");
    for (int i = 0; i < n; i++)
        if (d[i] == INF)
            printf("%d: Unreachable\n", i);
        else
            printf("%d: %d\n", i, d[i]);

    return 0;
}
```

OUTPUT :

```
Enter vertices and edges: 4 4
Enter edges (u v weight):
0 1 4
0 2 1
2 1 2
1 3 3
Enter source vertex: 0
Shortest distances:
0: 0
1: 3
2: 1
3: 6
```

5) DAG for topological sorting.

```
#include <stdio.h>

int graph[20][20], indegree[20], n;
```

```
int main() {
    int e, u, v, q[100], front = 0, rear = 0;
    int order[20], count = 0;

    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &e);

    printf("Enter directed edges (u v):\n");
    for (int i = 0; i < e; i++) {
        scanf("%d%d", &u, &v);
        graph[u][v] = 1;
        indegree[v]++;
    }

    for (int i = 0; i < n; i++)
        if (indegree[i] == 0)
            q[rear++] = i;

    while (front < rear) {
        u = q[front++];
        order[count++] = u;

        for (int v = 0; v < n; v++) {
            if (graph[u][v]) {
                indegree[v]--;
                if (indegree[v] == 0)
                    q[rear++] = v;
            }
        }
    }

    if (count != n) {
        printf("Graph contains a cycle");
    } else {
        printf("Topological order: ");
        for (int i = 0; i < count; i++)
            printf("%d ", order[i]);
    }

    return 0;
```

```
}
```

OUTPUT :

```
Enter vertices and edges: 6 6
Enter directed edges (u v):
0 1
0 2
1 3
2 3
2 4
3 5
Topological order: 0 1 2 4 3 5
```

6) Insert 76, 93, 40, 47, 10 and 55 in a hash table of size 7 using linear probing method.

```
#include <stdio.h>

#define SIZE 7

int main() {
    int table[SIZE];

    for (int i = 0; i < SIZE; i++)
        table[i] = -1;

    int keys[] = {76, 93, 40, 47, 10, 55};
    int n = 6;

    for (int i = 0; i < n; i++) {
        int key = keys[i];
        int index = key % SIZE;
        int start = index;

        while (table[index] != -1) {
            index = (index + 1) % SIZE;

            if (index == start) {
                printf("Hash table is full\n");
                break;
```

```
            }
        }

        if (table[index] == -1)
            table[index] = key;
    }

    printf("Hash Table:\n");
    for (int i = 0; i < SIZE; i++) {
        if (table[i] == -1)
            printf("%d : Empty\n", i);
        else
            printf("%d : %d\n", i, table[i]);
    }

    return 0;
}
```

OUTPUT :

```
Hash Table:
0 : 93
1 : 40
2 : 47
3 : 10
4 : 55
5 : Empty
6 : 76
```

7) Consider a hashtable size = 7, hash function as hash(x) = x%7 and collision resolution strategy to be f(i) = i^2. Insert 22, 30 and 50.

```
#include <stdio.h>

#define SIZE 7

int main() {
    int table[SIZE];
```

```
    for (int i = 0; i < SIZE; i++)
        table[i] = -1;

    int keys[] = {22, 30, 50};

    for (int k = 0; k < 3; k++) {
        int key = keys[k];
        int h = key % SIZE;
        int inserted = 0;

        for (int i = 0; i < SIZE; i++) {
            int index = (h + i * i) % SIZE;

            if (table[index] == -1) {
                table[index] = key;
                inserted = 1;
                break;
            }
        }

        if (!inserted)
            printf("Could not insert %d\n", key);
    }

    printf("Hash Table:\n");
    for (int i = 0; i < SIZE; i++) {
        if (table[i] == -1)
            printf("%d : Empty\n", i);
        else
            printf("%d : %d\n", i, table[i]);
    }

    return 0;
}
```

OUTPUT :

```
Hash Table:
0 : Empty
1 : 22
2 : 30
```

3 : Empty
4 : Empty
5 : 50
6 : Empty

8) Insert the keys 27, 43, 69, 2 into hash table of size 7, where first hash function is h1(k) = k mod 7 and second hash-function is h2(k) = 1 + (k mod 5) using double hashing technique.

```
#include <stdio.h>

#define SIZE 7

int main() {
    int table[SIZE];

    for (int i = 0; i < SIZE; i++)
        table[i] = -1;

    int keys[] = {27, 43, 69, 2};

    for (int i = 0; i < 4; i++) {
        int key = keys[i];
        int h1 = key % SIZE;
        int h2 = 1 + (key % 5);
        int inserted = 0;

        for (int j = 0; j < SIZE; j++) {
            int index = (h1 + j * h2) % SIZE;

            if (table[index] == -1) {
                table[index] = key;
                inserted = 1;
                break;
            }
        }
```

```
        if (!inserted)
            printf("Could not insert %d\n", key);
    }

    printf("Hash Table:\n");
    for (int i = 0; i < SIZE; i++) {
        if (table[i] == -1)
            printf("%d : Empty\n", i);
        else
            printf("%d : %d\n", i, table[i]);
    }

    return 0;
}
```

OUTPUT :
Hash Table:
0 : 43
1 : 69
2 : 2
3 : Empty
4 : Empty
5 : 27
6 : Empty

9) “If Quadratic probing is used and the table size is prime, then a new element can always be inserted if the table is at least half empty” .
Prove the above statement.

In quadratic probing, the positions checked are

hi(k)=(h(k)+i^2)mod m

where m is the table size.

If m is an odd prime, the values \(i^2 \bmod m\) for $i = 0, 1, \ldots, (m - 1)/2$ are distinct. Hence, quadratic probing examines $(m + 1)/2$ distinct positions.

If the table is at least half empty, fewer than or equal to half of its positions are occupied. Therefore, among the positions examined, at least one position must be empty.

Hence, a new element can be inserted.

10) Implement a hash data structure and write initialize fields and insert routine for separate chaining hash table creation.

```
#include <stdio.h>
#include <stdlib.h>

#define SIZE 10

struct Node {
    int data;
    struct Node *next;
};

struct Node *table[SIZE];

void initialize() {
    for (int i = 0; i < SIZE; i++)
        table[i] = NULL;
}

void insert(int key) {
    int index = key % SIZE;

    struct Node *n = malloc(sizeof(struct Node));
    if (n == NULL) {
        printf("Memory allocation failed\n");
        return;
    }

    n->data = key;
    n->next = table[index];
    table[index] = n;
```

```
}

void display() {
    for (int i = 0; i < SIZE; i++) {
        struct Node *p = table[i];

        printf("%d: ", i);
        while (p != NULL) {
            printf("%d -> ", p->data);
            p = p->next;
        }
        printf("NULL\n");
    }
}

int main() {
    initialize();

    insert(25);
    insert(15);
    insert(35);
    insert(12);
    insert(22);

    printf("Hash Table using Separate Chaining:\n");
    display();

    return 0;
}
```

OUTPUT :

```
Hash Table using Separate Chaining:
0: NULL
1: NULL
2: 22 -> 12 -> NULL
3: NULL
4: NULL
5: 35 -> 15 -> 25 -> NULL
6: NULL
7: NULL
```

8: NULL
9: NULL